

Python BDD Restful Automation

Assignment 2: Use of test data driven Automation in Python Behave Framework

1. Problem Statement: -

Adapt Python BDD framework for API automation / test data driven approach using Python Behave Framework.

2. Objective: -

Perform Python automations using Python Behave Framework focusing on a test data-driven approach.

3. Tools, Software, and Concepts Used: -

Python, Behave, Pycharm Community software.

4. Source Code / Implementation: -

```
features/data_driven.feature:-
```

```
Feature: Data Driven Login Testing
```

```
Scenario Outline: Attempt login with multiple credentials
```

```
    Given the user is on the login portal
```

```
    When the user inputs "<username>" and "<password>"
```

```
    Then the system should display "<expected_status>"
```

```
Examples:
```

username	password	expected_status	
standard_user	secret_sauce	success	
locked_out	secret_sauce	error	

```
steps/data_driven_steps.py:-
```

```
from behave import given, when, then
```

```
from selenium import webdriver
```

```
from selenium.webdriver.common.by import By
```

```
@given('the user is on the login portal')
```

```
def step_impl(context):
```

```
    context.driver = webdriver.Chrome()
```

```
    context.driver.get('https://www.saucedemo.com/')
```

```
@when('the user inputs "{user}" and "{pwd}"')
```

```
def step_impl(context, user, pwd):
```

```
    context.driver.find_element(By.ID, 'user-name').send_keys(user)
```

```
    context.driver.find_element(By.ID, 'password').send_keys(pwd)
```

```

    context.driver.find_element(By.ID, 'login-button').click()
@then('the system should display "{status}"')
def step_impl(context, status):
    if status == 'success':
        assert 'inventory' in context.driver.current_url
    else:
        error = context.driver.find_element(By.CSS_SELECTOR, '.error-
message-container').text
        assert len(error) > 0
    context.driver.quit()

```

5. Output & Screenshots: -

```

PS D:\Wipro Selenium> behave features/data_driven.feature
Feature: Data Driven Login Testing # features/data_driven.feature:1

  Then the system should display "success" # features/steps/data_driven_steps.py:16 2.207s
  Then the system should display "success" # features/steps/data_driven_steps.py:16
  Then the system should display "error" # features/steps/data_driven_steps.py:16 2.198s
  Then the system should display "error" # features/steps/data_driven_steps.py:16
1 feature passed, 0 failed, 0 skipped
2 scenarios passed, 0 failed, 0 skipped
6 steps passed, 0 failed, 0 skipped

```

Fig.1: Terminal output confirming the successful execution of a Data-Driven Behave test with two passed scenarios.

6. Result, Observation, and Conclusion: -

- Result: The Scenario Outline executed the same test logic multiple times with different data variables injected into the steps.
- Observation: Behave's Example tables automatically handle parameterization, removing the need for explicit loops in the Python code.
- Conclusion: Data-driven BDD ensures maximum test coverage across various inputs while maintaining a single, clean test script.